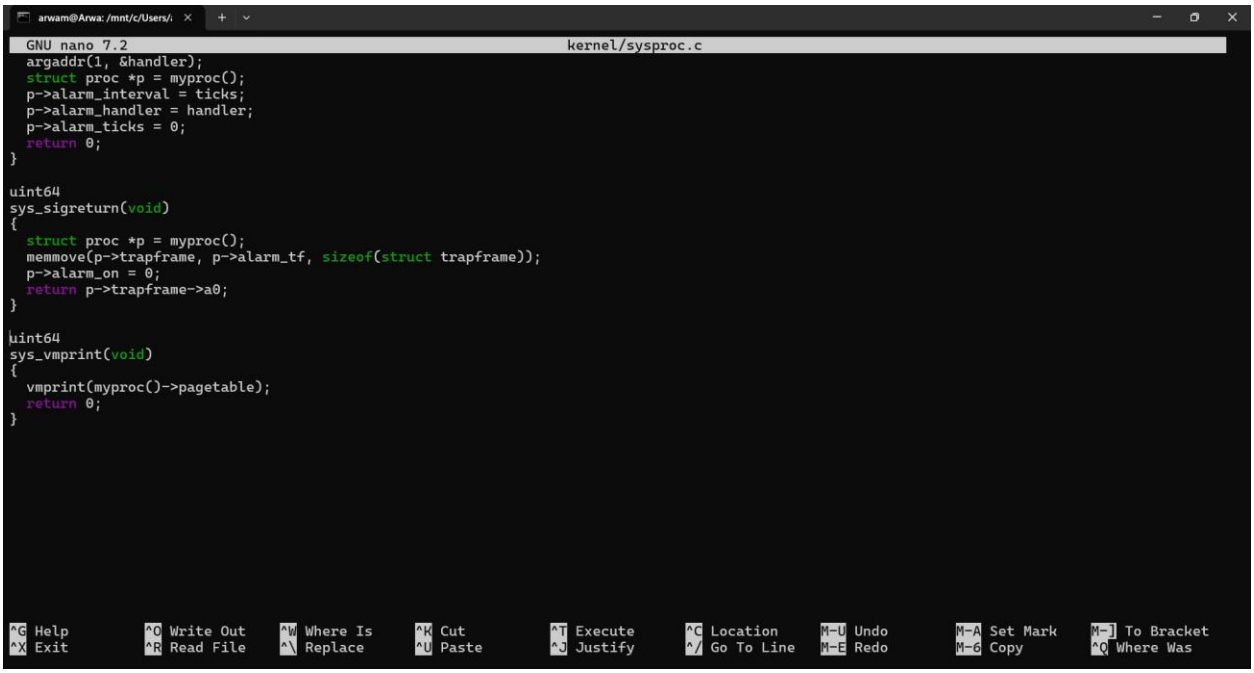
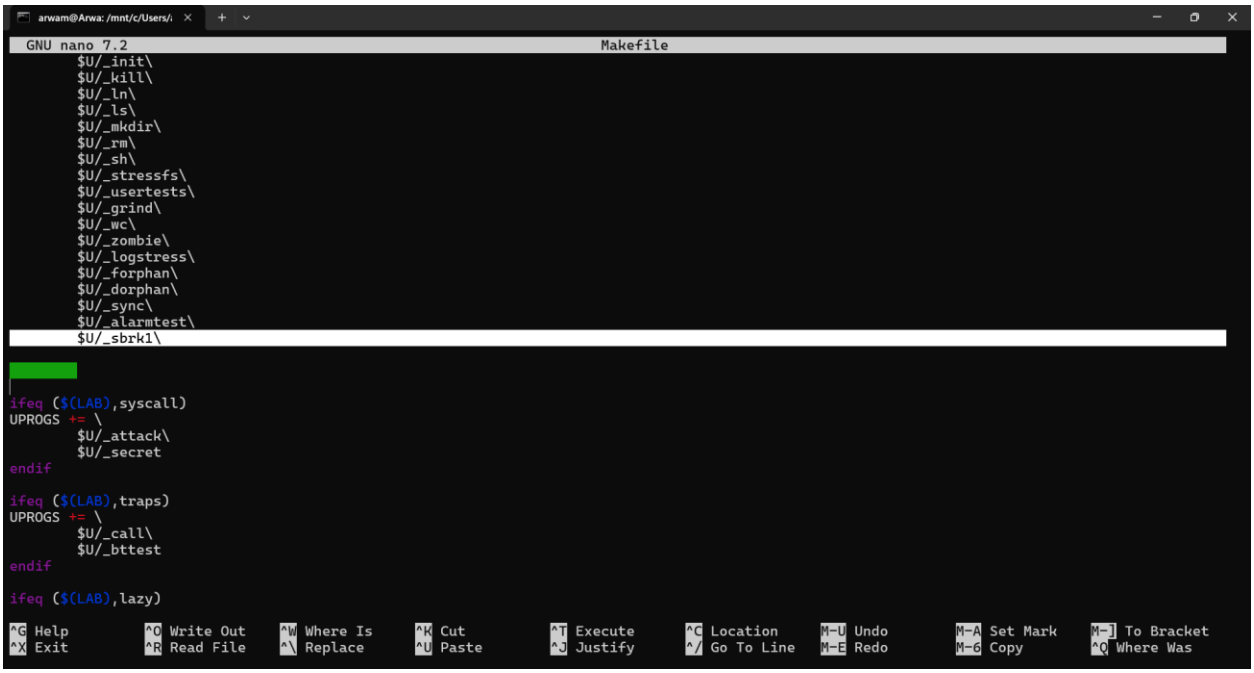
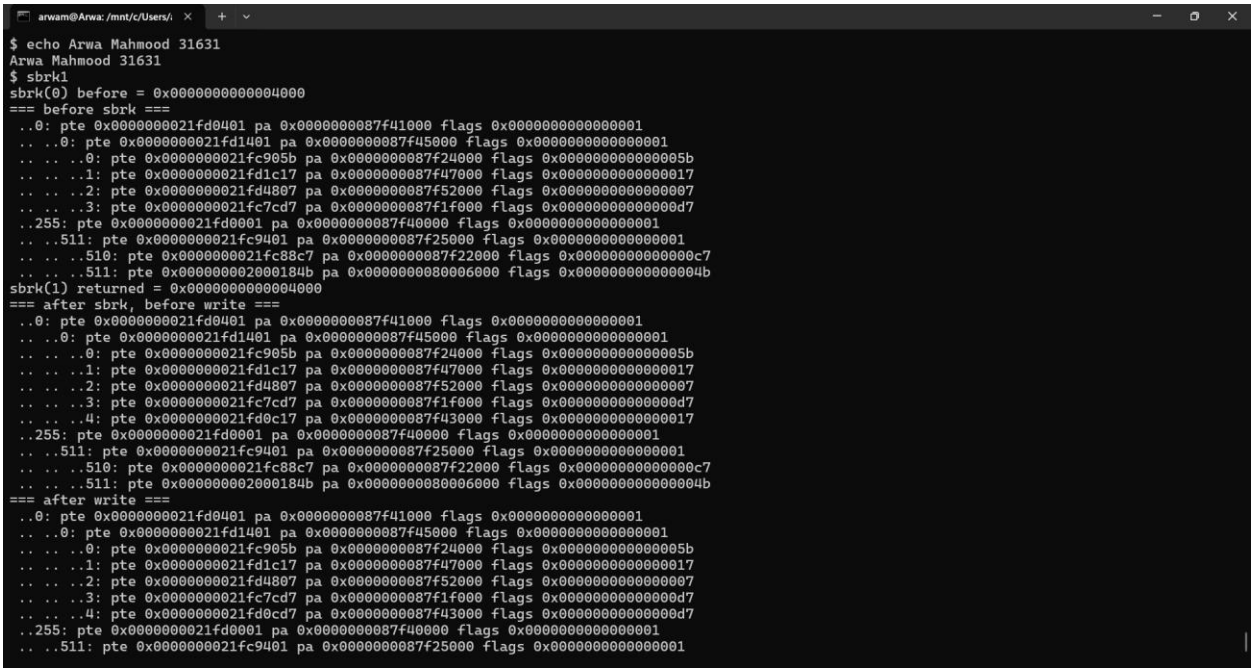
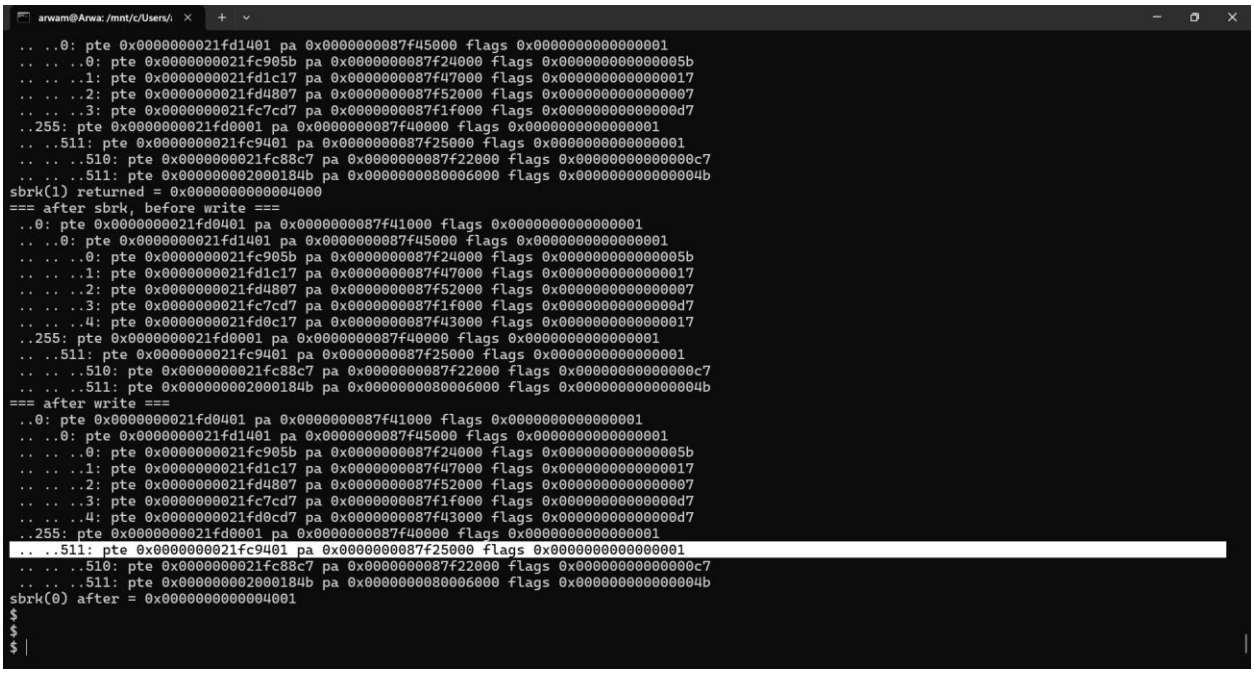
- **1.** Write a user program that grows its address space by one byte by calling `sbrk(1)`. Run the program and investigate the page table for the program before the call to `sbrk` and after the call to `sbrk`. How much space has the kernel allocated? What does the PTE for the new memory contain?






arwam@Arwa: /mnt/c/Users/
GNU nano 7.2 Makefile
$U/_init\
$U/_kill\
$U/_ln\
$U/_ls\
$U/_mkdir\
$U/_rm\
$U/_sh\
$U/_stressfs\
$U/_usertests\
$U/_grind\
$U/_wc\
$U/_zombie\
$U/_logstress\
$U/_forphan\
$U/_dorphan\
$U/_sync\
$U/_alarmtest\
$U/_sbrk1\
ifeq ($(LAB),syscall)
UPROGS += \
$U/_attack\
$U/_secret
endif
ifeq ($(LAB),traps)
UPROGS += \
$U/_call\
$U/_bttest
endif
ifeq ($(LAB),lazy)
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was


arwam@Arwa: /mnt/c/Users/
$ echo Arwa Mahmood 31631
Arwa Mahmood 31631
$ sbrk1
sbrk(0) before = 0x0000000000004000
=== before sbrk ===
..0: pte 0x0000000021fd0401 pa 0x0000000087f41000 flags 0x0000000000000001
.. ..0: pte 0x0000000021fd1401 pa 0x0000000087f45000 flags 0x0000000000000001
.. .. ..0: pte 0x0000000021fc905b pa 0x0000000087f24000 flags 0x000000000000005b
.. .. ..1: pte 0x0000000021fd1c17 pa 0x0000000087f47000 flags 0x0000000000000017
.. .. ..2: pte 0x0000000021fd4807 pa 0x0000000087f52000 flags 0x0000000000000007
.. .. ..3: pte 0x0000000021fc7cd7 pa 0x0000000087f1f000 flags 0x00000000000000d7
..255: pte 0x0000000021fd0001 pa 0x0000000087f40000 flags 0x0000000000000001
.. ..511: pte 0x0000000021fc9401 pa 0x0000000087f25000 flags 0x0000000000000001
.. .. ..510: pte 0x0000000021fc88c7 pa 0x0000000087f22000 flags 0x00000000000000c7
.. .. ..511: pte 0x000000002000184b pa 0x0000000080006000 flags 0x000000000000004b
sbrk(1) returned = 0x0000000000004000
=== after sbrk, before write ===
..0: pte 0x0000000021fd0401 pa 0x0000000087f41000 flags 0x0000000000000001
.. ..0: pte 0x0000000021fd1401 pa 0x0000000087f45000 flags 0x0000000000000001
.. .. ..0: pte 0x0000000021fc905b pa 0x0000000087f24000 flags 0x000000000000005b
.. .. ..1: pte 0x0000000021fd1c17 pa 0x0000000087f47000 flags 0x0000000000000017
.. .. ..2: pte 0x0000000021fd4807 pa 0x0000000087f52000 flags 0x0000000000000007
.. .. ..3: pte 0x0000000021fc7cd7 pa 0x0000000087f1f000 flags 0x00000000000000d7
.. .. ..4: pte 0x0000000021fd0c17 pa 0x0000000087f43000 flags 0x0000000000000017
..255: pte 0x0000000021fd0001 pa 0x0000000087f40000 flags 0x0000000000000001
.. ..511: pte 0x0000000021fc9401 pa 0x0000000087f25000 flags 0x0000000000000001
.. .. ..510: pte 0x0000000021fc88c7 pa 0x0000000087f22000 flags 0x00000000000000c7
.. .. ..511: pte 0x000000002000184b pa 0x0000000080006000 flags 0x000000000000004b
=== after write ===
..0: pte 0x0000000021fd0401 pa 0x0000000087f41000 flags 0x0000000000000001
.. ..0: pte 0x0000000021fd1401 pa 0x0000000087f45000 flags 0x0000000000000001
.. .. ..0: pte 0x0000000021fc905b pa 0x0000000087f24000 flags 0x000000000000005b
.. .. ..1: pte 0x0000000021fd1c17 pa 0x0000000087f47000 flags 0x0000000000000017
.. .. ..2: pte 0x0000000021fd4807 pa 0x0000000087f52000 flags 0x0000000000000007
.. .. ..3: pte 0x0000000021fc7cd7 pa 0x0000000087f1f000 flags 0x00000000000000d7
.. .. ..4: pte 0x0000000021fd0cd7 pa 0x0000000087f43000 flags 0x00000000000000d7
..255: pte 0x0000000021fd0001 pa 0x0000000087f40000 flags 0x0000000000000001
.. ..511: pte 0x0000000021fc9401 pa 0x0000000087f25000 flags 0x0000000000000001



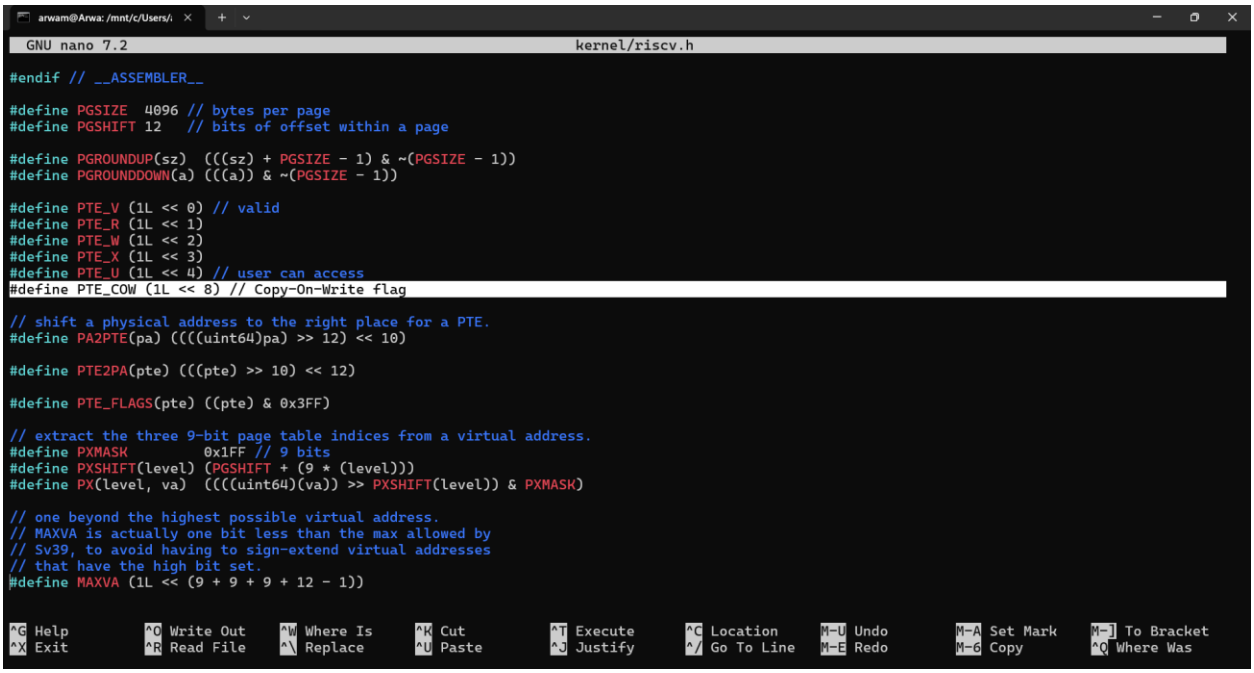
- **How much space has the kernel allocated?**

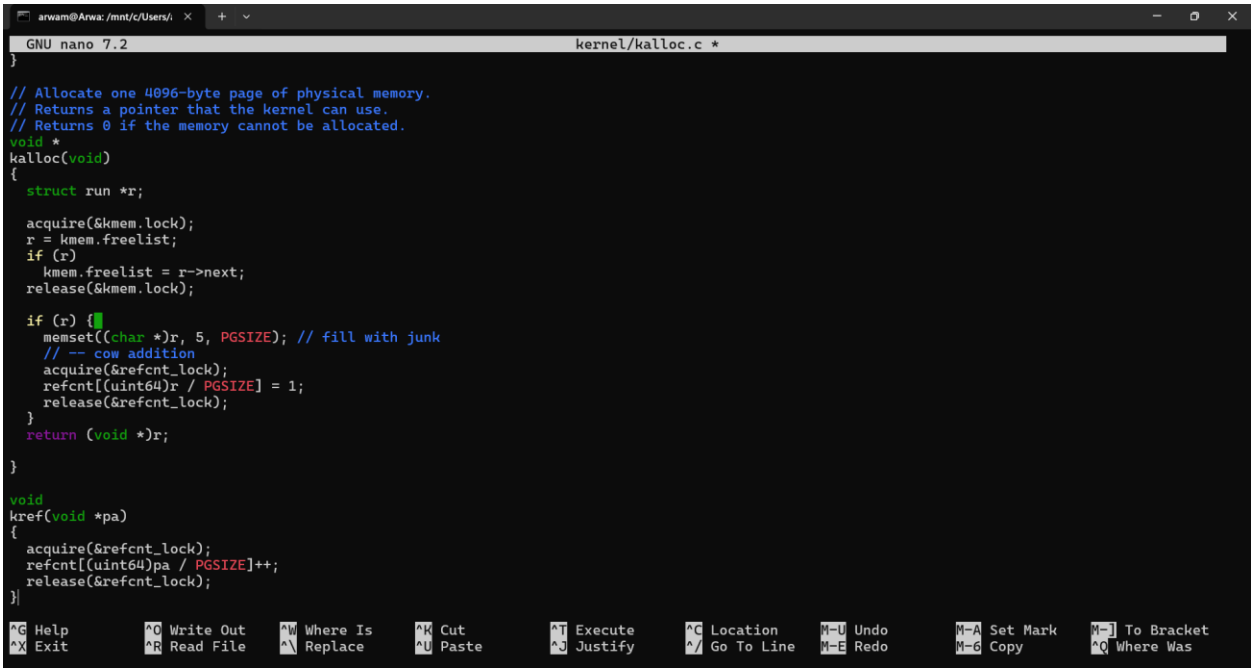
Ans. Even though sbrk(1) only increased the logical process size by 1 byte (from 0x4000 to 0x4001), the kernel physically allocated 4096 bytes (1 full page). This can be seen because the PTE was created for 0x4000.

- **What does the PTE for the new memory contain?**

Ans. It contains the Physical Page Number (PPN) pointing to a newly allocated memory page. Before writing, the flags were 0x17 (PTE_V | PTE_R | PTE_W | PTE_U). After writing to it, the flags became 0xd7 (PTE_V | PTE_R | PTE_W | PTE_U | PTE_A | PTE_D), showing that the hardware marked the page as Accessed and Dirty.

➢ 2. Cow Fork


arwam@Arwa: /mnt/c/Users/
GNU nano 7.2 kernel/riscv.h
#endif // __ASSEMBLER__
#define PGSIZE 4096 // bytes per page
#define PGSHIFT 12 // bits of offset within a page
#define PGROUNDUP(sz) (((sz) + PGSIZE - 1) & ~(PGSIZE - 1))
#define PGROUNDDOWN(a) (((a)) & ~(PGSIZE - 1))
#define PTE_V (1L << 0) // valid
#define PTE_R (1L << 1)
#define PTE_W (1L << 2)
#define PTE_X (1L << 3)
#define PTE_U (1L << 4) // user can access
#define PTE_COW (1L << 8) // Copy-On-Write flag
// shift a physical address to the right place for a PTE.
#define PA2PTE(pa) ((((uint64)pa) >> 12) << 10)
#define PTE2PA(pte) (((pte) >> 10) << 12)
#define PTE_FLAGS(pte) ((pte) & 0x3FF)
// extract the three 9-bit page table indices from a virtual address.
#define PXMASK 0x1FF // 9 bits
#define PXSHIFT(level) (PGSHIFT + (9 * (level)))
#define PX(level, va) ((((uint64)(va)) >> PXSHIFT(level)) & PXMASK)
// one beyond the highest possible virtual address.
// MAXVA is actually one bit less than the max allowed by
// Sv39, to avoid having to sign-extend virtual addresses
// that have the high bit set.
#define MAXVA (1L << (9 + 9 + 9 + 12 - 1))
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was


arwam@Arwa: /mnt/c/Users/
GNU nano 7.2 kernel/kalloc.c *
}
// Allocate one 4096-byte page of physical memory.
// Returns a pointer that the kernel can use.
// Returns 0 if the memory cannot be allocated.
void *
kalloc(void)
{
struct run *r;
acquire(&kmem.lock);
r = kmem.freelist;
if (r)
kmem.freelist = r->next;
release(&kmem.lock);
if (r) {
memset((char *)r, 5, PGSIZE); // fill with junk
// -- cow addition
acquire(&refcnt_lock);
refcnt[(uint64)r / PGSIZE] = 1;
release(&refcnt_lock);
}
return (void *)r;
}
void
kref(void *pa)
{
acquire(&refcnt_lock);
refcnt[(uint64)pa / PGSIZE]++;
release(&refcnt_lock);
}
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was

arwam@Arwa: /mnt/c/Users/i

GNU nano 7.2 kernel/defs.h *

```
struct inode*   ialloc(uint, short);
struct inode*   idup(struct inode*);
void            iinit();
void            ilock(struct inode*);
void            iput(struct inode*);
void            iunlock(struct inode*);
void            iunlockput(struct inode*);
void            iupdate(struct inode*);
int             namecmp(const char*, const char*);
struct inode*   namei(char*);
struct inode*   nameiparent(char*, char*);
int             readi(struct inode*, int, uint64, uint, uint);
void            stati(struct inode*, struct stat*);
int             writei(struct inode*, int, uint64, uint, uint);
void            itrunc(struct inode*);
void            ireclaim(int);

// kalloc.c
void*           kalloc(void);
void            kfree(void *);
void            kinit(void);
void            kref(void *);

// log.c
void            initlog(int, struct superblock*);
void            log_write(struct buf*);
void            begin_op(void);
void            end_op(void);

// pipe.c
int             pipealloc(struct file**, struct file**);
void            pipeclose(struct pipe*, int);
int             piperead(struct pipe*, uint64, int);
int             pipewrite(struct pipe*, uint64, int);
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was

arwam@Arwa: /mnt/c/Users/i

GNU nano 7.2 kernel/vm.c *

```
#include "param.h"
#include "types.h"
#include "memlayout.h"
#include "elf.h"
#include "riscv.h"
#include "defs.h"
#include "spinlock.h"
#include "proc.h"
#include "fs.h"


extern struct spinlock refcnt_lock;
extern int refcnt[];

/*
 * the kernel's page table.
 */
pagetable_t kernel_pagetable;

extern char etext[]; // kernel.ld sets this to end of kernel code.

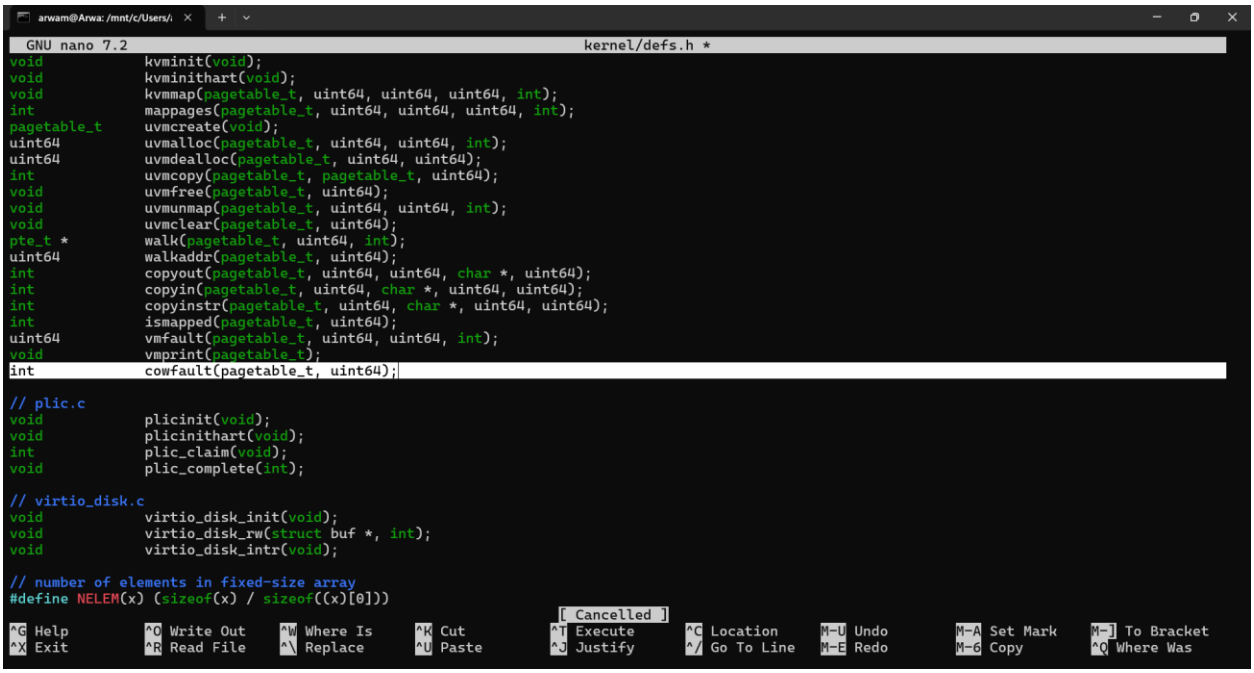
extern char trampoline[]; // trampoline.S

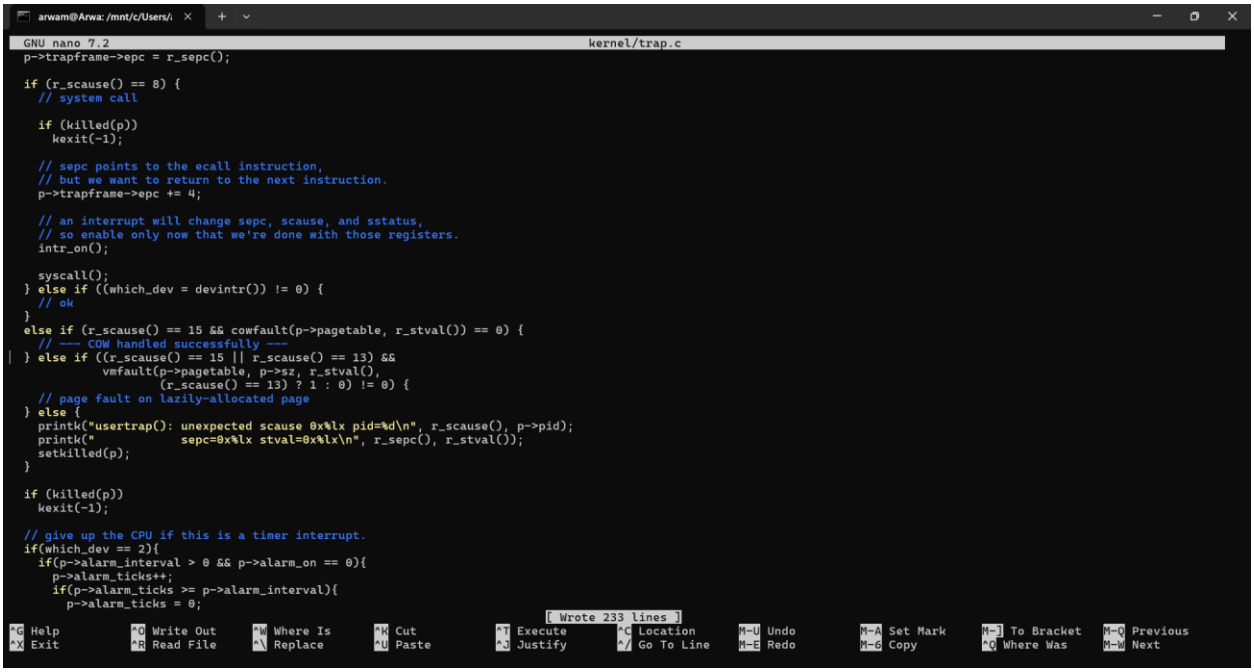
// Make a direct-map page table for the kernel.
pagetable_t
kvmmake(void)
{
  pagetable_t kpgtbl;

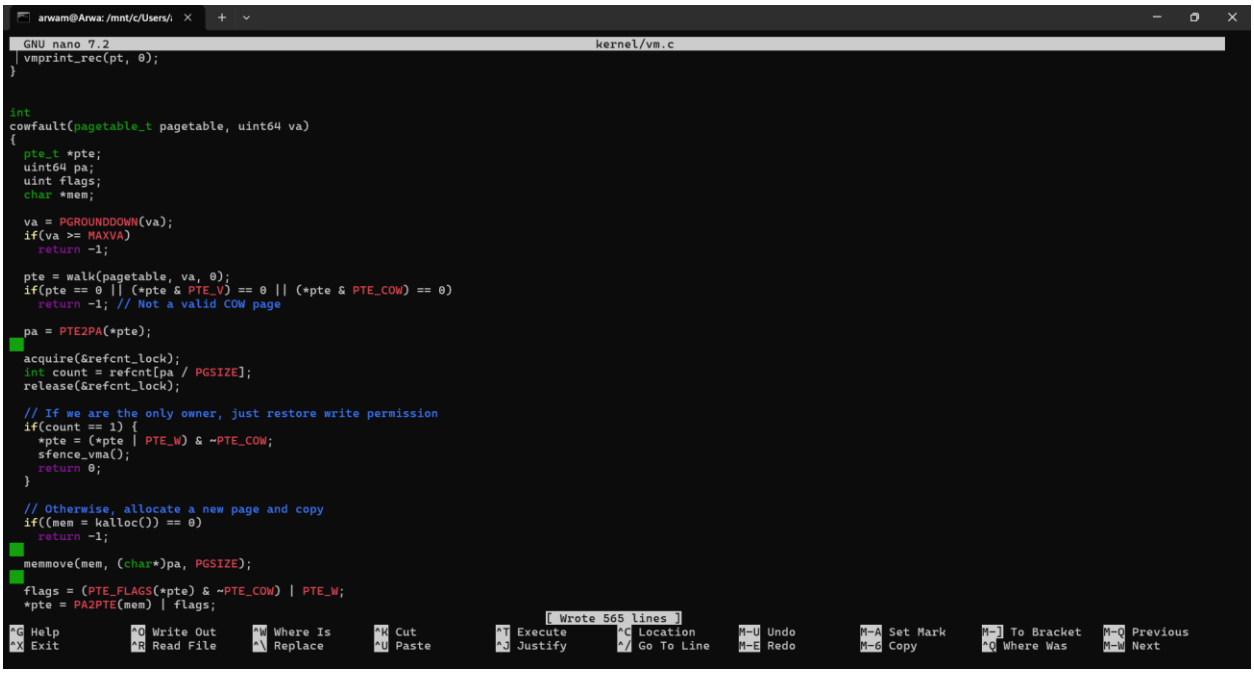
  kpgtbl = (pagetable_t)kalloc();
  memset(kpgtbl, 0, PGSIZE);

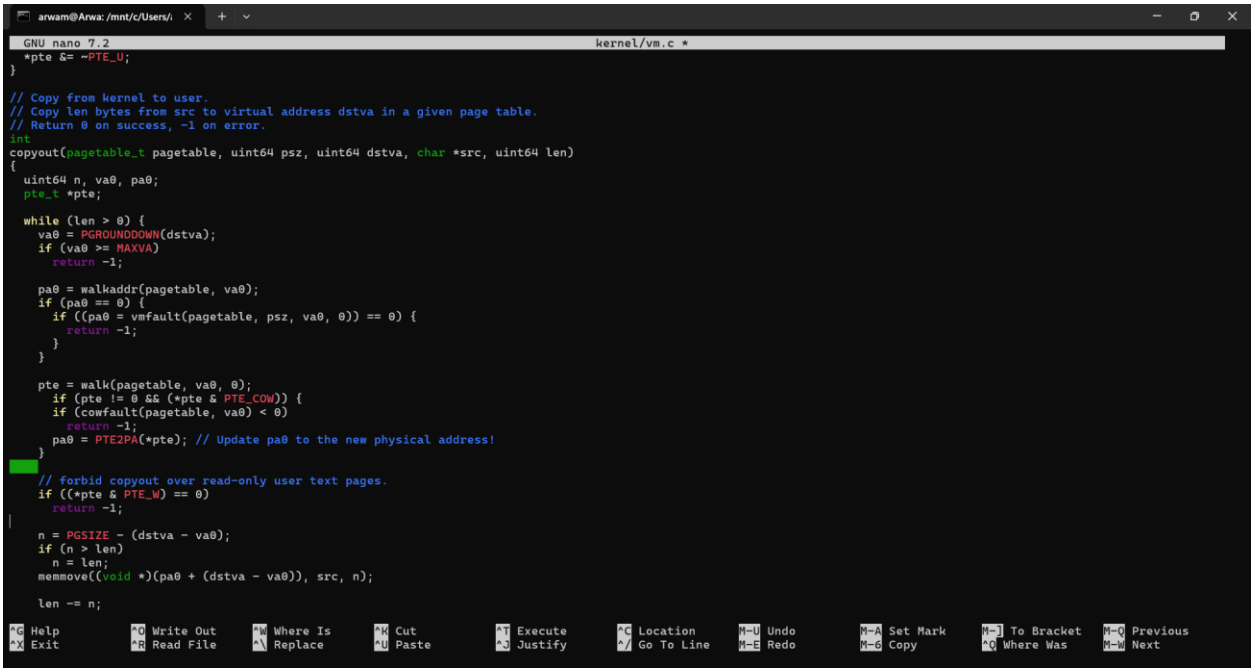
  // uart registers
  kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);
```

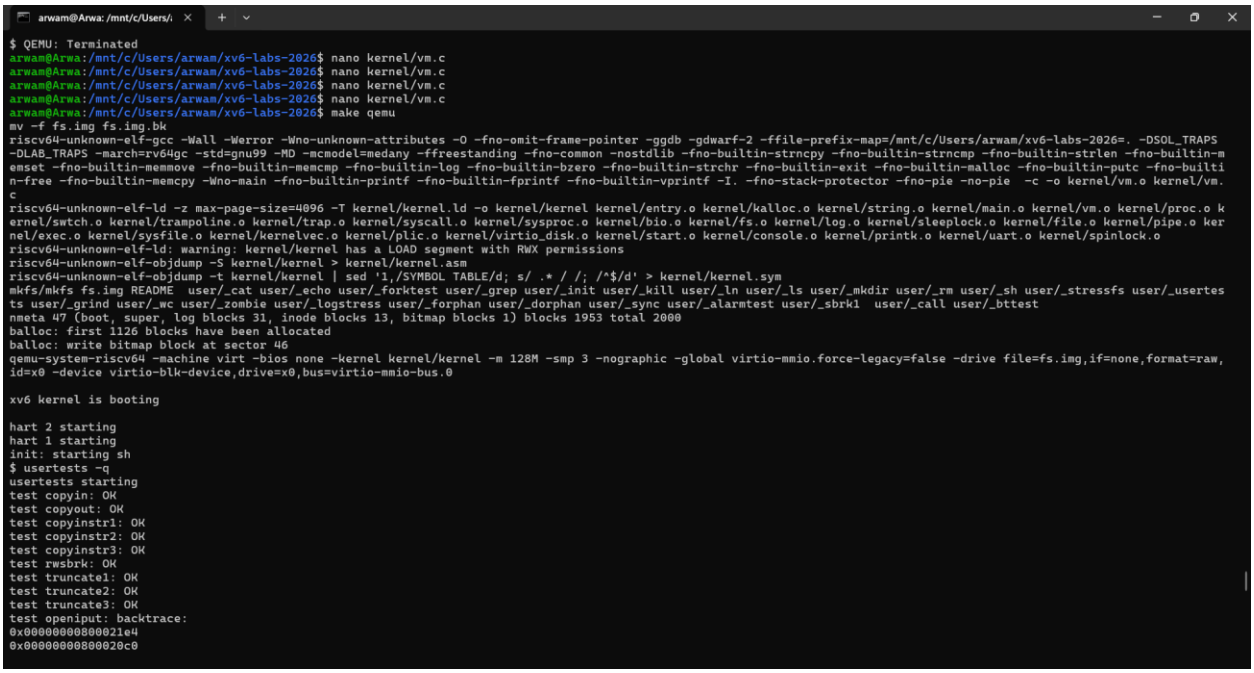
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was

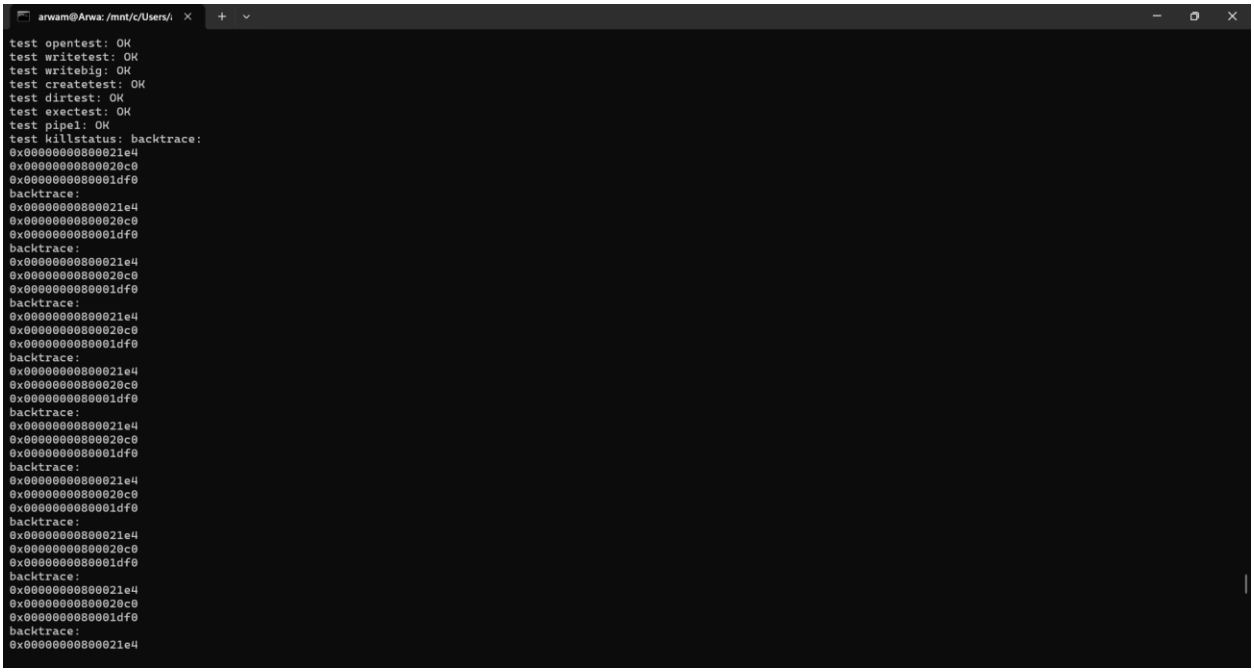

arwam@Arwa: /mnt/c/Users/i
GNU nano 7.2 kernel/defs.h *
void kvminit(void);
void kvminithart(void);
void kvmmap(pagetable_t, uint64, uint64, uint64, int);
int mappages(pagetable_t, uint64, uint64, uint64, int);
pagetable_t uvmcreate(void);
uint64 uvmalloc(pagetable_t, uint64, uint64, int);
uint64 uvmdealloc(pagetable_t, uint64, uint64);
int uvmcopy(pagetable_t, pagetable_t, uint64);
void uvmfree(pagetable_t, uint64);
void uvmunmap(pagetable_t, uint64, uint64, int);
void uvmclear(pagetable_t, uint64);
pte_t * walk(pagetable_t, uint64, int);
uint64 walkaddr(pagetable_t, uint64);
int copyout(pagetable_t, uint64, uint64, char *, uint64);
int copyin(pagetable_t, uint64, char *, uint64);
int copyinstr(pagetable_t, uint64, char *, uint64, uint64);
int ismapped(pagetable_t, uint64);
uint64 vmfault(pagetable_t, uint64, uint64, int);
void vmprint(pagetable_t);
int cowfault(pagetable_t, uint64);
// plic.c
void plicinit(void);
void plicinithart(void);
int plic_claim(void);
void plic_complete(int);
// virtio_disk.c
void virtio_disk_init(void);
void virtio_disk_rw(struct buf *, int);
void virtio_disk_intr(void);
// number of elements in fixed-size array
#define NELEM(x) (sizeof(x) / sizeof((x)[0]))
[Cancelled]
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was

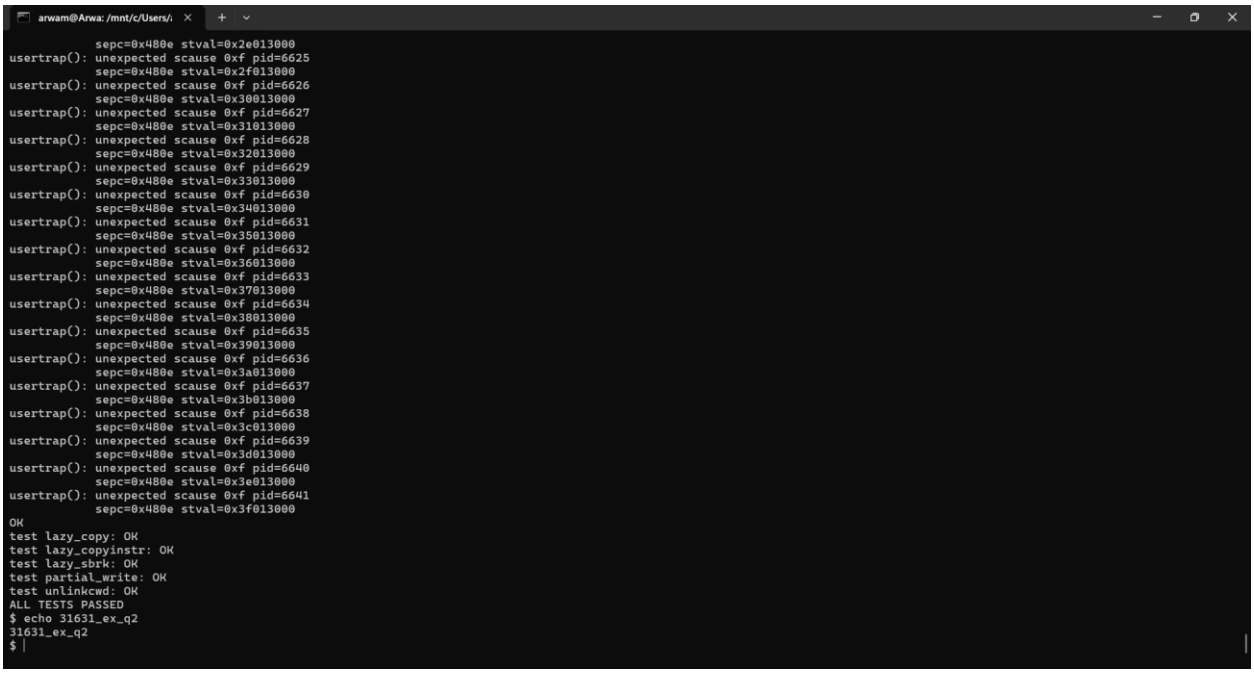

arwam@Arwa: /mnt/c/Users/i
GNU nano 7.2 kernel/trap.c
p->trapframe->epc = r_sepc();
if (r_scause() == 8) {
// system call
if (killed(p))
kexit(-1);
// sepc points to the ecall instruction,
// but we want to return to the next instruction.
p->trapframe->epc += 4;
// an interrupt will change sepc, scause, and sstatus,
// so enable only now that we're done with those registers.
intr_on();
syscall();
} else if ((which_dev = devintr()) != 0) {
// ok
}
else if (r_scause() == 15 && cowfault(p->pagetable, r_stval()) == 0) {
// --- COW handled successfully ---
} else if ((r_scause() == 15 || r_scause() == 13) &&
vmfault(p->pagetable, p->sz, r_stval(),
(r_scause() == 13) ? 1 : 0) != 0) {
// page fault on lazily-allocated page
} else {
printf("usertrap(): unexpected scause 0x%lx pid=%d\n", r_scause(), p->pid);
printf(" sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
setkilled(p);
}
if (killed(p))
kexit(-1);
// give up the CPU if this is a timer interrupt.
if(which_dev == 2){
if(p->alarm_interval > 0 && p->alarm_on == 0){
p->alarm_ticks++;
if(p->alarm_ticks >= p->alarm_interval){
p->alarm_ticks = 0;
[Wrote 233 lines]
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next


arwam@Arwa: /mnt/c/Users/
GNU nano 7.2 kernel/vm.c
vmprint_rec(pt, 0);
}
int
cowfault(pagetable_t pagetable, uint64 va)
{
pte_t *pte;
uint64 pa;
uint flags;
char *mem;
va = PGROUNDDOWN(va);
if(va >= MAXVA)
return -1;
pte = walk(pagetable, va, 0);
if(pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_COW) == 0)
return -1; // Not a valid COW page
pa = PTE2PA(*pte);
acquire(&refcnt_lock);
int count = refcnt[pa / PGSIZE];
release(&refcnt_lock);
// If we are the only owner, just restore write permission
if(count == 1) {
*pte = (*pte | PTE_W) & ~PTE_COW;
sfence_vma();
return 0;
}
// Otherwise, allocate a new page and copy
if((mem = kalloc()) == 0)
return -1;
memmove(mem, (char*)pa, PGSIZE);
flags = (PTE_FLAGS(*pte) & ~PTE_COW) | PTE_W;
*pte = PA2PTE(mem) | flags;
[Wrote 565 lines]
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next


arwam@Arwa: /mnt/c/Users/
GNU nano 7.2 kernel/vm.c *
*pte &= ~PTE_U;
}
// Copy from kernel to user.
// Copy len bytes from src to virtual address dstva in a given page table.
// Return 0 on success, -1 on error.
int
copyout(pagetable_t pagetable, uint64 psz, uint64 dstva, char *src, uint64 len)
{
uint64 n, va0, pa0;
pte_t *pte;
while (len > 0) {
va0 = PGROUNDDOWN(dstva);
if (va0 >= MAXVA)
return -1;
pa0 = walkaddr(pagetable, va0);
if (pa0 == 0) {
if ((pa0 = vmfault(pagetable, psz, va0, 0)) == 0) {
return -1;
}
}
pte = walk(pagetable, va0, 0);
if (pte != 0 && (*pte & PTE_COW)) {
if (cowfault(pagetable, va0) < 0)
return -1;
pa0 = PTE2PA(*pte); // Update pa0 to the new physical address!
}
// forbid copyout over read-only user text pages.
if ((*pte & PTE_W) == 0)
return -1;
n = PGSIZE - (dstva - va0);
if (n > len)
n = len;
memmove((void *)(pa0 + (dstva - va0)), src, n);
len -= n;
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next


arwam@Arwa: /mnt/c/Users/a
$ QEMU: Terminated
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026$ nano kernel/vm.c
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026$ nano kernel/vm.c
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026$ nano kernel/vm.c
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026$ nano kernel/vm.c
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026$ make qemu
mv -f fs.img fs.img.bk
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/mnt/c/Users/arwam/xv6-labs-2026=. -DSOL_TRAPS -DLAB_TRAPS -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/vm.o kernel/vm.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/swtch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/start.o kernel/console.o kernel/printk.o kernel/uart.o kernel/spinlock.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync user/_alarmtest user/_sbrk1 user/_call user/_bttest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1126 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ usertests -q
usertests starting
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test copyinstr3: OK
test rwsbrk: OK
test truncate1: OK
test truncate2: OK
test truncate3: OK
test openiput: backtrace:
0x00000008800021e4
0x00000008800020c0


arwam@Arwa: /mnt/c/Users/a
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test dirtest: OK
test exectest: OK
test pipe1: OK
test killstatus: backtrace:
0x00000008800021e4
0x00000008800020c0
0x0000000880001df0
backtrace:
0x00000008800021e4
0x00000008800020c0
0x0000000880001df0
backtrace:
0x00000008800021e4
0x00000008800020c0
0x0000000880001df0
backtrace:
0x00000008800021e4
0x00000008800020c0
0x0000000880001df0
backtrace:
0x00000008800021e4
0x00000008800020c0
0x0000000880001df0
backtrace:
0x00000008800021e4
0x00000008800020c0
0x0000000880001df0
backtrace:
0x00000008800021e4
0x00000008800020c0
0x0000000880001df0
backtrace:
0x00000008800021e4
0x00000008800020c0
0x0000000880001df0
backtrace:
0x00000008800021e4


arwam@Arwa: /mnt/c/Users/
sepc=0x480e stval=0x2e013000
usertrap(): unexpected scause 0xf pid=6625
sepc=0x480e stval=0x2f013000
usertrap(): unexpected scause 0xf pid=6626
sepc=0x480e stval=0x30013000
usertrap(): unexpected scause 0xf pid=6627
sepc=0x480e stval=0x31013000
usertrap(): unexpected scause 0xf pid=6628
sepc=0x480e stval=0x32013000
usertrap(): unexpected scause 0xf pid=6629
sepc=0x480e stval=0x33013000
usertrap(): unexpected scause 0xf pid=6630
sepc=0x480e stval=0x34013000
usertrap(): unexpected scause 0xf pid=6631
sepc=0x480e stval=0x35013000
usertrap(): unexpected scause 0xf pid=6632
sepc=0x480e stval=0x36013000
usertrap(): unexpected scause 0xf pid=6633
sepc=0x480e stval=0x37013000
usertrap(): unexpected scause 0xf pid=6634
sepc=0x480e stval=0x38013000
usertrap(): unexpected scause 0xf pid=6635
sepc=0x480e stval=0x39013000
usertrap(): unexpected scause 0xf pid=6636
sepc=0x480e stval=0x3a013000
usertrap(): unexpected scause 0xf pid=6637
sepc=0x480e stval=0x3b013000
usertrap(): unexpected scause 0xf pid=6638
sepc=0x480e stval=0x3c013000
usertrap(): unexpected scause 0xf pid=6639
sepc=0x480e stval=0x3d013000
usertrap(): unexpected scause 0xf pid=6640
sepc=0x480e stval=0x3e013000
usertrap(): unexpected scause 0xf pid=6641
sepc=0x480e stval=0x3f013000
OK
test lazy_copy: OK
test lazy_copyinstr: OK
test lazy_sbrk: OK
test partial_write: OK
test unlinkcwd: OK
ALL TESTS PASSED
$ echo 31631_ex_q2
31631_ex_q2
$